Seeded Reflection Power

A Proposed Contender 7 for BusyLean

Research package prepared for Vladimir Reshetnikov

October 12, 2026

Abstract

We propose a new closed, constructive Lean natural number that strictly exceeds BusyLean's current Contender 6 without using the incumbent value as an iteration counter. The key observation is that the existing forest-reflection scheduler has a reusable higher-order type: it maps an oracle family and a W-tree control object to a natural-valued function. We define an order-raising operator that reflects an *entire scheduler* through the repository's universal W-tree index. Iterating this operator yields a strict hierarchy of schedulers. The final number uses a promotion height equal to the P5 seed value at 42, with a small successor guard, and evaluates the resulting scheduler at the same universal index and input 42.

The supplied Lean development is designed to prove

$$\texttt{BusyLean.contender_6} < \texttt{BusyLean.contender_7},$$

as well as the inherited comparisons with the exact Contender 5 port and P5. All recursion is structural or primitive, and the defining closure is intended to avoid every named incumbent value. The archive includes Lean source, a kernel-side closure audit, a patch, a pinned CI workflow, executable finite models, static audits, this article, and a validation ledger. The present environment did not contain a Lean executable, so the package does not claim kernel verification; it records that limitation explicitly and supplies the exact commands needed to complete the check under Lean 4.34.0.

Verification status. The mathematical construction and proof term have been written against the APIs in BusyLean commit `86b360648163f0249eab79539438b6a83dfe8215`. Static audits and executable finite analogues pass. The Lean files have *not* been kernel-checked in this environment. No statement below should be read as a claim of completed Lean verification until `lake build` and the supplied audit file both succeed on the pinned toolchain.

Contents

1 Executive summary **3**

2 Repository baseline and constraints **3**
2.1 Audited revision 3
2.2 Contest constraints relevant to the proposal 4

3 The incumbent reflector **5**
3.1 Families and visible maxima 5
3.2 Forest scheduling 5

4 Scheduler-level reflection **5**
4.1 The scheduler type . . . 5
4.2 One order lift . . . 6
4.3 Finite orders . . . 6

5 The proposed number **6**
5.1 A seeded height . . . 6
5.2 Contender 7 . . . 7
5.3 Why the number is computable in principle . . . 7

6 Formal comparison architecture **7**
6.1 The domination invariant . . . 7
6.2 Base scheduler . . . 7
6.3 A child is below its promoted parent . . . 8
6.4 Leaf lift below universal-index lift . . . 8
6.5 The invariant is preserved . . . 8
6.6 Strict growth at every successor order . . . 9
6.7 The required comparison . . . 9
6.8 Inherited comparisons . . . 10

7 Magnitude analysis **10**
7.1 What is formal . . . 10
7.2 Repository calibration . . . 10
7.3 Effect of one scheduler lift . . . 10
7.4 Why this is more than “use the old number as a counter” . . . 11

8 Engineering analysis **11**
8.1 Expected definition-check behavior . . . 11
8.2 Expected proof-check behavior . . . 11
8.3 Axiom and dependency audit . . . 12
8.4 Why no monotonicity lemma appears . . . 12
8.5 Integration footprint . . . 12

9 Independent executable checks **12**
9.1 Static source audit . . . 12
9.2 Finite reflection-power model . . . 13
9.3 Pinned CI workflow . . . 13

10 Risks and unresolved verification points **13**
10.1 Lean elaboration risk . . . 13
10.2 Timing risk . . . 13
10.3 Calibration risk . . . 14
10.4 Judging-policy risk . . . 14

11 Further research questions **14**

12 Conclusion **15**

A Complete defining module **15**

B Complete comparison module **16**

C Kernel audit module **18**

D Reproduction commands **19**

1 Executive summary

The incumbent has the form

$$C_6 = \mathrm{stage}(\mathrm{p5Family}, I, 42), \tag{1}$$

where I is a universal W-tree assembled from all trees denoted by the closed programs of P5's W-language. The repository informally calibrates this near $f_{\Lambda^2}(42)$, with $\Lambda \approx \psi(\Omega_\omega)$.

The new construction promotes the scheduler itself. If S is a scheduler, its lift $\mathcal{L}S$ recursively traverses a second W-tree. At a leaf it runs S through I. At a node it recursively lifts every child and exposes those values to S as a new oracle family, again running S through I. Let

$$S_0 = \mathrm{stage}, \qquad S_{k+1} = \mathcal{L}S_k.$$

Let

$$P = \mathrm{p5Family}(0, 42), \qquad H = \mathrm{succ}(42 + P).$$

The proposed number is

$$C_7 = S_H(\mathrm{p5Family}, I, 42). \tag{2}$$

The formal comparison rests on a single invariant:

$$\mathrm{Dom}(S) : \Longleftrightarrow \ \forall a \, \forall i \le n, \quad a(i, n) < S(a, I, n).$$

The existing scheduler has this property. If S has it, then $\mathcal{L}S$ has it and, at I, strictly exceeds S. Therefore

$$S_0(a, I, n) < S_1(a, I, n) < S_2(a, I, n) < \cdots .$$

Since $C_6 = S_0(\mathrm{p5Family}, I, 42)$ and $H > 0$, the required strict inequality follows.

The important qualitative points are these:

1. The proposal introduces a genuine order-raising operation; it is not only a larger decimal constant.
2. The final height is huge but constructive and independent of the named incumbent value.
3. The proof never assumes monotonicity of reflection in its oracle. That monotonicity is false in general because reflected programs may inspect oracle values.
4. The huge height remains symbolic in the comparison proof. Kernel checking should not evaluate P.

2 Repository baseline and constraints

2.1 Audited revision

The development targets BusyLean commit `86b360648163f0249eab79539438b6a83dfe8215`. The repository pins `leanprover/lean4:v4.34.0`. At that revision the relevant modules are:

`Reflection.lean`	Defines oracle evaluation, finite-prefix reflection, and W-tree scheduling.
`ReflectionFacts.lean`	Proves visible oracle values, visible programs, seed values, and child stages are strictly exceeded.
`Contender6.lean`	Defines the universal tree index, the P5 seed family, the final family, and the incumbent.
`Comparison.lean`	Proves the incumbent exceeds P5 and the exact Contender 5 port, plus structural domination results.

The incumbent definitions reduce to:

```
def p5Family : Family :=
  fun _ n => BigNumber.iterate BigNumber.diagonal n n

def finalFamily (n : Nat) : Nat :=
  stage p5Family index n

def contender_6 : Nat := finalFamily 42
```

The universal index is

```
def treesOfDepth (k : Nat) : WTree Nat :=
  .node fun i => defTree (nthD (BigNumber.allTerms k) i .zero)

def index : WTree Nat := .node treesOfDepth
```

The first tree under the first depth join is definitionally a leaf:

```
theorem first_tree_leaf :
  defTree (nthD (BigNumber.allTerms 0) 0 .zero) = .leaf := rfl
```

This two-edge spine,

$$\text{leaf} \longrightarrow \text{treesOfDepth}(0) \longrightarrow I,$$

will be reused at every scheduler order.

2.2 Contest constraints relevant to the proposal

The proposal is designed around the following repository requirements.

1. The number must be a closed, computable `Nat`.
2. No partial, unsafe, external, noncomputable, or oracle stand-in is permitted.
3. A theorem must strictly compare the new number with the incumbent.
4. The defining closure must not smuggle in the incumbent value as a huge iteration counter.
5. Definition and proof checking must remain symbolic enough to fit the contest's build limits.
6. The formal argument must respect the repository's constructive axiom policy.

The construction below was selected partly because it addresses all six at the level of its architecture. In particular, the comparison theorem unfolds only a successor constructor around the height; it never asks Lean to compute the height.

3 The incumbent reflector

3.1 Families and visible maxima

Write

$$\mathcal{F} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

for the type called `Family`. For $a \in \mathcal{F}$, the repository's `reflect` function at input n includes the finite prefix

$$\max_{i \le n} a(i, n)$$

and the maximum value of every enumerated object-language term of depth at most n, evaluated with a as an oracle family. It then adds $n + 1$. Consequently the repository proves

$$i \le n \quad \Longrightarrow \quad a(i, n) < \mathrm{reflect}(a, n). \tag{3}$$

The crucial point is that Equation (3) is a direct maximum inequality. It does not require `reflect` to be monotone in a. Such monotonicity would be unsafe to assume: a program can compare or otherwise inspect oracle values and follow a different branch when the oracle changes.

3.2 Forest scheduling

For a W-tree $t \in \mathcal{W} = \mathrm{WTree}(\mathbb{N})$, the incumbent scheduler is

$$\mathrm{stage}(a, \mathrm{leaf}, n) = \mathrm{reflect}(a, n),$$

$$\mathrm{stage}(a, \mathrm{node}(f), n) = \mathrm{reflect}\big((i, m) \mapsto \mathrm{stage}(a, f(i), m), n\big).$$

Equation (3) immediately gives two facts already proved in `ReflectionFacts.lean`:

$$a(i, n) < \mathrm{stage}(a, \mathrm{leaf}, n) \qquad (i \le n), \tag{4}$$

$$\mathrm{stage}(a, f(i), n) < \mathrm{stage}(a, \mathrm{node}(f), n) \qquad (i \le n). \tag{5}$$

Applying these along the two-edge spine inside I proves that the incumbent final family exceeds the P5 seed family. The new proposal repeats the same local reasoning one type level higher, with schedulers replacing ordinary stages.

4 Scheduler-level reflection

4.1 The scheduler type

A scheduler has type

$$\mathsf{Scheduler} = \mathcal{F} \to \mathcal{W} \to \mathbb{N} \to \mathbb{N}.$$

This is exactly the Lean abbreviation:

```
abbrev StageOperator := Family → WTree Nat → Nat → Nat
```

The incumbent `stage` is one such scheduler. The second argument need not be interpreted only as an object-level ordinal control. It can also index a family of recursively promoted schedulers.

4.2 One order lift

Definition 4.1 (Scheduler lift)**.** For a scheduler S, seed family a, W-tree u, and input n, define

$$(\mathcal{L}S)(a, \mathrm{leaf}, n) = S(a, I, n),$$

and

$$(\mathcal{L}S)(a, \mathrm{node}(f), n) = S\big((i, m) \mapsto (\mathcal{L}S)(a, f(i), m), I, n\big).$$

The Lean definition is almost a transliteration:

```
def liftStage (S : StageOperator) (seed : Family) :
    WTree Nat → Nat → Nat
  | .leaf => S seed index
  | .node f =>
      S (fun i => liftStage S seed (f i)) index
```

The recursive call is on the immediate W-subtree $f(i)$, just as in the repository's existing `stage`. There is no general recursion and no termination proof beyond the W-tree recursor.

Remark 4.2. The second W-tree in $\mathcal{L}S$ controls *operator nesting*. At a leaf, we run the entire previous scheduler through the universal object-level index. At a node, the recursively promoted child operators become the oracle family seen by the previous scheduler. Thus one lift is not merely one more object stage; it reflects the scheduler's complete behavior.

4.3 Finite orders

Definition 4.3 (Reflection-power hierarchy)**.** Let

$$S_0 = \mathrm{stage}, \qquad S_{k+1} = \mathcal{L}S_k.$$

In Lean:

```
def stageOrder : Nat → StageOperator
  | 0 => stage
  | k + 1 => liftStage (stageOrder k)
```

This is primitive recursion on `Nat`. For every concrete natural k, S_k is a total scheduler.

5 The proposed number

5.1 A seeded height

Let

$$P = \mathrm{p5Family}(0, 42) = \mathrm{iterate}(\mathrm{diagonal}, 42, 42).$$

This is definitionally the numerical expression used by P5's proposal, but the new definition refers to the seed family rather than to a named contender constant. Define

$$H = \mathrm{succ}(42 + P). \tag{6}$$

The successor wrapper has two purposes. It gives a large positive height and, more importantly for the comparison proof, exposes positivity by one constructor without normalizing P.

The exact Lean definition is:

```
def metaHeight : Nat := Nat.succ (42 + p5Family 0 42)
```

5.2 Contender 7

Construction 5.1 (Seeded reflection power)**.** Define

$$C_7 = S_H(\text{p5Family}, I, 42).$$

Equivalently:

```
def contender_7 : Nat :=
  stageOrder metaHeight p5Family index 42
```

This definition is intentionally compact. The large number comes from the normalization behavior of a small total program, not from a large source file or an embedded numeral.

5.3 Why the number is computable in principle

Proposition 5.2. *C_7 is a well-defined computable natural number.*

Proof. The P5 interpreter, term enumeration, finite maximum, diagonal function, `reflect`, and `stage` are total definitions already present in the repository. The new function `liftStage` is structural recursion on an inductive W-tree. The function `stageOrder` is primitive recursion on a natural number. The height H is a closed application of total functions, and every remaining argument in Construction 5.1 is closed. Therefore normalization terminates in the metatheory, although the required resources are incomprehensibly large. □

Caution 5.3. "Computable in principle" does not mean feasible. Evaluating C_7, or even materializing more than tiny toy instances of the hierarchy, is not a practical objective. The contest compares closed normal forms by proof, not by execution.

6 Formal comparison architecture

This section gives the complete mathematical proof mirrored by `Comparison7.lean`.

6.1 The domination invariant

Definition 6.1. A scheduler S is *index-dominating* if

$$\text{Dom}(S) : \Longleftrightarrow\ \forall a \in \mathcal{F}\ \forall i, n \in \mathbb{N}, \quad i \le n \Longrightarrow a(i,n) < S(a, I, n).$$

The Lean predicate is:

```
def IndexDominates (S : StageOperator) : Prop :=
  ∀ (seed : Family) {i n : Nat},
    i ≤ n → seed i n < S seed index n
```

6.2 Base scheduler

Lemma 6.2. *The incumbent scheduler $S_0 = \text{stage}$ is index-dominating.*

Proof. Fix a, i, n with $i \le n$. By Equation (4),

$$a(i,n) < \text{stage}(a, \text{leaf}, n).$$

The leaf is the zeroth child of treesOfDepth(0), and the latter is the zeroth child of I. Applying Equation (5) twice gives

$$\mathrm{stage}(a, \mathrm{leaf}, n) < \mathrm{stage}(a, \mathrm{treesOfDepth}(0), n) < \mathrm{stage}(a, I, n).$$

Transitivity proves the claim. □

The Lean proof reuses `first_tree_leaf` exactly as the incumbent comparison proof does. It is generic in the seed family, which is what makes the higher-order induction possible.

6.3 A child is below its promoted parent

Lemma 6.3. *Assume* $\mathrm{Dom}(S)$. *If* $i \le n$, *then*

$$(\mathcal{L}S)(a, f(i), n) < (\mathcal{L}S)(a, \mathrm{node}(f), n).$$

Proof. Unfold the node clause of Definition 4.1. Put

$$b(j, m) = (\mathcal{L}S)(a, f(j), m).$$

The right side becomes $S(b, I, n)$, while the left side is $b(i, n)$. The claim is precisely $\mathrm{Dom}(S)$ applied to b and $i \le n$. □

This is the key higher-order step. It uses only the domination invariant; it does not compare two unrelated oracle families.

6.4 Leaf lift below universal-index lift

Lemma 6.4. *If* $\mathrm{Dom}(S)$, *then for every* a, n,

$$(\mathcal{L}S)(a, \mathrm{leaf}, n) < (\mathcal{L}S)(a, I, n).$$

Proof. Apply Lemma 6.3 along the same two-edge spine used in Lemma 6.2:

$$(\mathcal{L}S)(a, \mathrm{leaf}, n) < (\mathcal{L}S)(a, \mathrm{treesOfDepth}(0), n) < (\mathcal{L}S)(a, I, n).$$

□

6.5 The invariant is preserved

Lemma 6.5. *If* $\mathrm{Dom}(S)$, *then* $\mathrm{Dom}(\mathcal{L}S)$.

Proof. For $i \le n$, first use $\mathrm{Dom}(S)$:

$$a(i, n) < S(a, I, n).$$

By the leaf equation in Definition 4.1,

$$S(a, I, n) = (\mathcal{L}S)(a, \mathrm{leaf}, n).$$

Lemma 6.4 then gives

$$a(i, n) < (\mathcal{L}S)(a, I, n).$$

□

Corollary 6.6. *Every scheduler order* S_k *is index-dominating.*

Proof. Induction on k, using Lemma 6.2 at zero and Lemma 6.5 at successors. □

6.6 Strict growth at every successor order

Theorem 6.7. *For every seed family a, input n, and order k,*

$$S_k(a, I, n) < S_{k+1}(a, I, n).$$

Proof. By the leaf equation,

$$S_k(a, I, n) = (\mathcal{L}S_k)(a, \mathrm{leaf}, n).$$

Corollary 6.6 gives $\mathrm{Dom}(S_k)$, so Lemma 6.4 yields

$$(\mathcal{L}S_k)(a, \mathrm{leaf}, n) < (\mathcal{L}S_k)(a, I, n) = S_{k+1}(a, I, n).$$

□

Corollary 6.8. *For every $h > 0$,*

$$S_0(a, I, n) < S_h(a, I, n).$$

The Lean development packages this as a recursion over a predecessor:

```
theorem stageOrder_zero_lt_succ (seed : Family) (n : Nat) :
    ∀ k,
      stageOrder 0 seed index n <
      stageOrder (Nat.succ k) seed index n
```

6.7 The required comparison

Theorem 6.9 (Proposed kernel theorem)**.**

$$\textit{BusyLean.contender_6} < \textit{BusyLean.contender_7}.$$

Proof. The incumbent is definitionally

$$C_6 = S_0(\mathrm{p5Family}, I, 42).$$

The proposed number is

$$C_7 = S_{\mathrm{succ}(42+P)}(\mathrm{p5Family}, I, 42).$$

Apply Corollary 6.8 with predecessor $42 + P$. □

The Lean proof is correspondingly short:

```
theorem contender_6_lt_contender_7 :
    contender_6 < contender_7 := by
  change stageOrder 0 p5Family index 42 <
    stageOrder (Nat.succ (42 + p5Family 0 42))
      p5Family index 42
  exact stageOrder_zero_lt_succ
    p5Family 42 (42 + p5Family 0 42)
```

No reduction of `p5Family 0 42` is required. The term appears only as a symbolic argument to a theorem quantified over every natural predecessor.

6.8 Inherited comparisons

Once Theorem 6.9 is accepted, the existing repository theorems give immediately:

$$\texttt{contender_5} < \texttt{contender_7}$$

and

$$\texttt{BigNumber.contender_6} < \texttt{contender_7}.$$

The latter name is P5's proposal-local constant, not the BusyLean incumbent. The archive includes both transitivity proofs.

7 Magnitude analysis

7.1 What is formal

Only the following magnitude claims are intended as formal Lean theorems:

1. every scheduler order is index-dominating;
2. each successor scheduler order is strictly larger at the universal index;
3. the incumbent is strictly below the proposed number;
4. the exact Contender 5 port and P5's proposal-local number are also below the proposed number by transitivity.

These are ordinary inequalities in $\mathbb{N}$. They do not require ordinal notations, well-ordering proofs, or a formal fast-growing hierarchy.

7.2 Repository calibration

For scale only, adopt the current repository article's notation

$$\Lambda \approx \psi(\Omega_\omega)$$

for the strength of P5's W-language. The repository estimates

$$P \quad \text{near} \quad f_{\Lambda+\omega^2}(42)$$

and the incumbent near

$$C_6 \quad \text{near} \quad f_{\Lambda\cdot\Lambda}(42) = f_{\Lambda^2}(42).$$

These are calibrations, not proved equalities.

7.3 Effect of one scheduler lift

A stage along I traverses an index whose available W-ranks are calibrated by Λ. A scheduler lift traverses a fresh copy of that index while each node invokes the complete preceding scheduler. The natural ordinal bookkeeping model is therefore right multiplication by Λ:

$$\alpha \longmapsto \alpha \cdot \Lambda.$$

Starting from Λ^2, this suggests

$$S_h(\text{p5Family}, I, 42) \quad \text{near} \quad f_{\Lambda^{h+2}}(42)$$

for finite h. At the seeded height H, the proposal is therefore heuristically

$$C_7 \quad \text{near} \quad f_{\Lambda^{H+2}}(42), \qquad H = \text{succ}(42 + P). \tag{7}$$

Even a fixed height of 42 would move the calibration from Λ^2 to roughly Λ^{44}. The chosen height is itself the P5 seed value plus a guard, producing an incomparably larger finite exponent.

Caution 7.1. Equation (7) is intentionally not advertised as a theorem. A rigorous calibration would require a formal ordinal semantics for the object language, the universal index, oracle interpretation, and scheduler lift. The finite exponent H is an actual natural number, however enormous; the claim is not that the construction reaches a literal transfinite iteration count.

7.4 Why this is more than "use the old number as a counter"

The construction has two independent sources of strength.

1. **New operation.** $S \mapsto \mathcal{L}S$ raises the order of reflection by recursively turning promoted child schedulers into the previous scheduler's oracle family. This operation already yields an infinite strict hierarchy when iterated through ordinary small naturals.

2. **Seeded scale.** The final order count is synchronized with the inherited P5 diagonal value. It is not the BusyLean incumbent and is not needed to establish the hierarchy's strictness; it only selects a very high member of the new hierarchy.

A conservative variant $S_{42}(\text{p5Family}, I, 42)$ is therefore available if a contest judge prefers a fixed promotion count. The main package uses the seeded height because the repository rules prohibit the incumbent value specifically, not all previously defined constructive growth functions, and because the strictness proof remains entirely symbolic.

8 Engineering analysis

8.1 Expected definition-check behavior

The defining module contains only an abbreviation and three short recursive definitions. Lean checks the structural equations for `liftStage` and the primitive equations for `stageOrder`; it need not normalize `metaHeight` or `contender_7`. The source therefore has a plausible path to the repository's definition-time limit.

8.2 Expected proof-check behavior

The proof introduces a generic invariant and proves it once. The final theorem applies a universally quantified lemma at the symbolic predecessor `42 + p5Family 0 42`. It does not construct a transitivity chain of that length and does not reduce the P5 term. Thus proof size is constant in the numerical value of the height.

8.3 Axiom and dependency audit

The supplied audit computes the constant closure of `BusyLean.contender_7`. It is intended to fail if that closure includes any of:

```
BusyLean.contender_6
BusyLean.contender_5
BigNumber.contender_6
BigNumber.contender_5
```

Reusing `p5Family`, `index`, and the total language implementation is allowed and visible. Reusing a named predecessor value as an opaque subterm is not.

The audit also runs `#print axioms` on the new number and comparison theorems. The expected axiom profile is inherited from the accepted upstream modules; the proposal adds no axiom declaration, choice operator, or external implementation hook.

8.4 Why no monotonicity lemma appears

It would be tempting to prove that a larger oracle produces a larger reflected value and then lift inequalities pointwise. The upstream source explicitly warns that this principle fails: interpreted programs may test oracle values. The present proof instead arranges for each desired child value to be a *visible component of the exact oracle family* passed to the parent. Equation (3) then gives a strict inequality directly. This is both more constructive and more robust.

8.5 Integration footprint

The overlay adds:

- `BusyLean/Contender7.lean`;
- `BusyLean/Comparison7.lean`;
- two imports in `BusyLean.lean`;
- `test/Contender7Audit.lean`.

No upstream file needs semantic modification. The archive contains a complete Git patch against the audited revision.

9 Independent executable checks

Because a Lean executable was unavailable, the package includes checks that are useful but explicitly weaker than kernel verification.

9.1 Static source audit

The static audit verifies:

1. absence of `noncomputable`, `partial`, `unsafe`, `extern`, `implemented_by`, `native_decide`, `sorry`, `admit`, and custom `axiom` declarations from the production overlay;
2. absence of the token `contender_6` from the defining module;
3. presence of the intended defining equations and all comparison lemmas;

4. balanced delimiters and the kernel closure test.

The check passes for the files in this archive.

9.2 Finite reflection-power model

A memoized Python model replaces infinitely branching W-trees by finite trees with a leaf default and retains the exact recursive architecture of `stageOrder`. Its reflector exceeds every visible oracle value, matching the only property used by the formal proof. It tests scheduler orders $0, 1, 2, 3$ and inputs $0, 1, 2$.

Input	Order 0	Order 1	Order 2	Order 3
0	6	12	30	84
1	14	38	134	518
2	24	60	204	780

Every row is strictly increasing. The model also checks, at every tested input and order, the exact proof spine

$$(\mathcal{L}S)(a, \text{leaf}, n) < (\mathcal{L}S)(a, \text{treesOfDepth}(0), n) < (\mathcal{L}S)(a, I, n).$$

This catches errors in recursion direction and oracle wiring, but it does not parse Lean and is not evidence about kernel acceptance.

9.3 Pinned CI workflow

The archive provides a GitHub Actions workflow that uses the repository's `lean-toolchain`, deletes `.lake`, runs `lake build`, and then runs the closure/axiom audit. This is the intended final acceptance test.

10 Risks and unresolved verification points

A serious proposal should distinguish mathematical risk from environment risk.

10.1 Lean elaboration risk

The source was written by matching the exact signatures of `stage_child_lt`, `seed_lt_stage_leaf`, `first_tree_leaf`, `treesOfDepth`, and `index` at the audited commit. Nevertheless, only the Lean elaborator and kernel can confirm all implicit arguments, definitional reductions, and equation-compiler behavior. The supplied code is compact enough that any elaboration repair should be local, but no repair is silently assumed here.

10.2 Timing risk

The design is intended to avoid normalization of huge terms, but the contest limits must be measured on the pinned toolchain from a clean build. The CI workflow should record wall-clock times for the defining module, comparison module, and audit separately before repository acceptance.

10.3 Calibration risk

The ordinal estimate depends on the repository's informal calibration and on the modeling choice $\alpha \mapsto \alpha\Lambda$ for one scheduler lift. A formal semantics might produce a different expression. This does not affect the strict natural-number comparison.

10.4 Judging-policy risk

A judge might prefer a fixed order such as 42 rather than a P5-scaled order, even though the stated anti-laziness rule bars the incumbent value rather than all large constructive counters. The new operator and all proofs remain valid under the fixed-height variant. The archive documents this fallback as a policy variant, not as the primary proposal.

11 Further research questions

R1. Kernel closure and timing. Does the overlay build unchanged on Lean 4.34.0, and what are the clean-build times of the definition, comparison, and audit modules?

R2. Formal ordinal semantics. Construct an ordinal notation system and interpretation for the P5 term language, `stage`, the universal index, and `liftStage`. Prove sound upper and lower bounds for each scheduler order rather than relying on the heuristic Λ^{h+2}.

R3. Fixed-height versus seeded-height policy. Determine a contest standard for large iteration counts inherited from earlier constructive languages. A formal "non-laziness margin" predicate could make this judgment objective.

R4. Operator-level universal trees. The present hierarchy iterates the same lift by a natural count. Can one define a conservative language of scheduler expressions whose own W-trees index heterogeneous sequences of order-raising operators, while retaining an internal normalization proof?

R5. Transfinite promotion without nontermination. Develop a notation system for well-founded trees of scheduler lifts and an evaluator by structural recursion. This could replace the finite count H with a directly represented well-founded control tree.

R6. System F and higher universes. The upstream roadmap proposes System F, System F_ω, and the Calculus of Constructions as qualitatively stronger base languages. How does scheduler reflection interact with each normalization proof? Does "stronger language first, then reflection power" dominate pure reflection iteration decisively?

R7. Parametric comparison theorem. Abstract the two-edge spine of `index` into a structure containing a distinguished leaf descendant. Prove the entire scheduler-power theorem for any such index and any base scheduler satisfying visible-oracle domination.

R8. Proof-term minimization. Can the comparison be expressed using a reusable algebra of progressive schedulers, reducing both source size and kernel-checking time while preserving a transparent closure audit?

R9. Machine-checked resource bounds. Formalize a syntactic cost model for elaboration and kernel checking, then prove that the comparison proof avoids evaluating `p5Family 0 42`.

R10. Independent implementation. Reimplement the scheduler-power construction in a minimal standalone Lean file and prove agreement with the repository overlay, analogous to the existing port-agreement audit.

12 Conclusion

The universal W-tree index in Contender 6 does more than support one forest reflection. Because the scheduler itself has the type of a family-to-tree transformer, the same index can promote the scheduler as an object. The resulting lift preserves a simple domination invariant and is strictly progressive at the universal index. Primitive recursion therefore gives a strict hierarchy whose zero level is exactly the incumbent scheduler.

Selecting the P5 seed value, plus a successor guard, as the hierarchy height produces a closed constructive number with a very large formal margin:

$$C_6 < C_7.$$

The proof is structurally small and does not normalize the huge height. The proposal is consequently a plausible next BusyLean contender even before the more ambitious language-strength jumps in the repository roadmap.

The remaining decisive step is mechanical rather than conceptual: run the provided overlay and audit under the exact Lean 4.34.0 toolchain, record timing, and accept or locally repair any elaboration issue. Until that is done, the archive should be treated as a rigorous research proposal with executable supporting evidence, not as a completed kernel-certified submission.

A Complete defining module

```
import BusyLean.Contender6

/-!
# Contender 7: seeded reflection power

The current `stage` scheduler performs forest reflection through a `WTree Nat`.
`liftStage` promotes an arbitrary scheduler `S` by using a second W-tree as an
operator-level control object.  A leaf runs `S` through the universal `index`;
a node feeds the promoted values of all children back to `S` as its oracle
family, again at `index`.

`stageOrder k` iterates this promotion `k` times.  The final height is the P5
seed value at 42, plus a fixed positive guard.  The definition does not use the
value `BusyLean.contender_6` as an iteration counter.

Do not evaluate `contender_7`.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- A forest-reflection scheduler: oracle family, control tree, input. -/
abbrev StageOperator := Family → WTree Nat → Nat → Nat
```

```
/-- Raise a scheduler by one reflection order.

At a leaf, run `S` with the supplied seed through the universal index.  At a
node, recursively promote every child and expose the resulting family to `S`.
All recursion is structural recursion on the W-tree. -/
def liftStage (S : StageOperator) (seed : Family) : WTree Nat → Nat → Nat
  | .leaf => S seed index
  | .node f => S (fun i => liftStage S seed (f i)) index

/-- The finite hierarchy of scheduler orders. -/
def stageOrder : Nat → StageOperator
  | 0 => stage
  | k + 1 => liftStage (stageOrder k)

/-- A large, closed, computable promotion height.

The successor guard makes positivity definitional, so the comparison proof
never needs to normalize the P5 value. -/
def metaHeight : Nat := Nat.succ (42 + p5Family 0 42)

/-- **Contender 7.**  Never evaluate it. -/
def contender_7 : Nat := stageOrder metaHeight p5Family index 42

end BusyLean
```

B Complete comparison module

```
import BusyLean.Contender7
import BusyLean.Comparison

/-!
# Contender 7: comparison theorems

The central invariant says that a scheduler, when run through the universal
`index`, exceeds every visible entry of its seed family.  The existing `stage`
satisfies the invariant.  One reflection-order lift preserves it and is
strictly larger at `index`; induction therefore gives a strict hierarchy.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- `S` exceeds every visible seed entry when scheduled through `index`. -/
def IndexDominates (S : StageOperator) : Prop :=
  ∀ (seed : Family) {i n : Nat}, i ≤ n → seed i n < S seed index n

/-- The existing forest scheduler dominates its seed at the universal index. -/
theorem seed_lt_stage_index (seed : Family) {i n : Nat} (hi : i ≤ n) :
    seed i n < stage seed index n := by
  have h1 : seed i n < stage seed .leaf n :=
    seed_lt_stage_leaf seed hi
  have h2 :
      stage seed (defTree (nthD (BigNumber.allTerms 0) 0 .zero)) n <
        stage seed (treesOfDepth 0) n :=
```

```
    stage_child_lt seed _ (Nat.zero_le n)
  have h3 : stage seed (treesOfDepth 0) n < stage seed index n :=
    stage_child_lt seed treesOfDepth (Nat.zero_le n)
  rw [first_tree_leaf] at h2
  exact Nat.lt_trans h1 (Nat.lt_trans h2 h3)

/-- The base scheduler has the domination invariant. -/
theorem stage_indexDominates : IndexDominates stage := by
  intro seed i n hi
  exact seed_lt_stage_index seed hi

/-- A promoted child is visible to, and strictly below, its promoted parent. -/
theorem liftStage_child_lt {S : StageOperator} (hS : IndexDominates S)
    (seed : Family) (f : Nat → WTree Nat) {i n : Nat} (hi : i ≤ n) :
    liftStage S seed (f i) n < liftStage S seed (.node f) n := by
  change liftStage S seed (f i) n <
    S (fun j => liftStage S seed (f j)) index n
  exact hS (fun j => liftStage S seed (f j)) hi

/-- For every invariant-preserving scheduler, its leaf lift is strictly below
its lift through the universal index. -/
theorem liftStage_leaf_lt_index {S : StageOperator} (hS : IndexDominates S)
    (seed : Family) (n : Nat) :
    liftStage S seed .leaf n < liftStage S seed index n := by
  have h1 :
      liftStage S seed (defTree (nthD (BigNumber.allTerms 0) 0 .zero)) n <
        liftStage S seed (treesOfDepth 0) n :=
    liftStage_child_lt hS seed _ (Nat.zero_le n)
  have h2 :
      liftStage S seed (treesOfDepth 0) n < liftStage S seed index n :=
    liftStage_child_lt hS seed treesOfDepth (Nat.zero_le n)
  rw [first_tree_leaf] at h1
  exact Nat.lt_trans h1 h2

/-- One order lift preserves the domination invariant. -/
theorem liftStage_indexDominates {S : StageOperator} (hS : IndexDominates S) :
    IndexDominates (liftStage S) := by
  intro seed i n hi
  have h0 : seed i n < S seed index n := hS seed hi
  have h0' : seed i n < liftStage S seed .leaf n := by
    change seed i n < S seed index n
    exact h0
  exact Nat.lt_trans h0' (liftStage_leaf_lt_index hS seed n)

/-- Every finite scheduler order retains the domination invariant. -/
theorem stageOrder_indexDominates : ∀ k, IndexDominates (stageOrder k)
  | 0 => stage_indexDominates
  | k + 1 => liftStage_indexDominates (stageOrder_indexDominates k)

/-- Every successor scheduler order is strictly larger at the universal index. -/
theorem stageOrder_step (seed : Family) (k n : Nat) :
    stageOrder k seed index n < stageOrder (k + 1) seed index n := by
  change liftStage (stageOrder k) seed .leaf n <
    liftStage (stageOrder k) seed index n
  exact liftStage_leaf_lt_index (stageOrder_indexDominates k) seed n

/-- Order zero is strictly below every positive order. -/
theorem stageOrder_zero_lt_succ (seed : Family) (n : Nat) :
    ∀ k, stageOrder 0 seed index n < stageOrder (Nat.succ k) seed index n
```

```
  | 0 => by
      simpa using stageOrder_step seed 0 n
  | k + 1 => by
      exact Nat.lt_trans (stageOrder_zero_lt_succ seed n k)
        (by simpa using stageOrder_step seed (k + 1) n)

/-- **Required comparison:** the current repository contender is strictly below
Contender 7.  The proof unfolds only the successor guard, not the huge P5
height. -/
theorem contender_6_lt_contender_7 : contender_6 < contender_7 := by
  change stageOrder 0 p5Family index 42 <
    stageOrder (Nat.succ (42 + p5Family 0 42)) p5Family index 42
  exact stageOrder_zero_lt_succ p5Family 42 (42 + p5Family 0 42)

/-- The exact Contender 5 port is also strictly below Contender 7. -/
theorem contender_5_lt_contender_7 : contender_5 < contender_7 :=
  Nat.lt_trans contender_5_lt_contender_6 contender_6_lt_contender_7

/-- Proposal P5's number is strictly below Contender 7. -/
theorem p5_lt_contender_7 : BigNumber.contender_6 < contender_7 :=
  Nat.lt_trans p5_lt_contender_6 contender_6_lt_contender_7

end BusyLean
```

C Kernel audit module

```
import BusyLean
import Lean

/-!
# Contender 7 kernel audit

Run with `lake env lean test/Contender7Audit.lean` after installing the overlay.
It prints axioms, proves that the new number's defining closure does not contain
any incumbent contender value, and records the comparison theorem signatures.
-/

open Lean Elab Command

/-- All constants reachable from `root` through declaration types and values. -/
def contender7ConstClosure (env : Environment) (root : Name) : NameSet := Id.run do
  let mut seen : NameSet := {}
  let mut stack := #[root]
  while stack.size > 0 do
    let c := stack.back!
    stack := stack.pop
    if seen.contains c then continue
    seen := seen.insert c
    let some ci := env.find? c | continue
    let body : Array Name := match ci with
      | .defnInfo v => v.value.getUsedConstants
      | .thmInfo v => v.value.getUsedConstants
      | .opaqueInfo v => v.value.getUsedConstants
      | _ => #[]
    stack := stack ++ ci.type.getUsedConstants ++ body
  return seen
```

```
/-- Fail if any listed constant occurs in the closure of the first one. -/
elab "#assert_not_in_contender7_closure " root:ident " : " bad:ident* : command => do
  let env ← getEnv
  let r ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo root
  let cl := contender7ConstClosure env r
  for b in bad do
    let n ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo b
    if cl.contains n then throwError m!"{r} depends on forbidden constant {n}"
  logInfo m!"{r}: closure of {cl.size} constants avoids all {bad.size} forbidden names"

#print axioms BusyLean.contender_7
#print axioms BusyLean.contender_6_lt_contender_7
#print axioms BusyLean.contender_5_lt_contender_7
#print axioms BusyLean.p5_lt_contender_7
#print axioms BusyLean.stageOrder_indexDominates

-- The construction reuses the incumbent language and seed, but not any named
-- incumbent value as an iteration counter or hidden subterm.
#assert_not_in_contender7_closure BusyLean.contender_7 :
  BusyLean.contender_6 BusyLean.contender_5
  BigNumber.contender_6 BigNumber.contender_5

#check @BusyLean.contender_6_lt_contender_7
#check @BusyLean.contender_5_lt_contender_7
#check @BusyLean.p5_lt_contender_7
#check @BusyLean.stageOrder_step
#check @BusyLean.stageOrder_indexDominates
```

D Reproduction commands

From a checkout of the audited BusyLean commit:

```
cp lean/BusyLean/Contender7.lean BusyLean/
cp lean/BusyLean/Comparison7.lean BusyLean/
cat lean/BusyLean.lean.append >> BusyLean.lean
cp lean/test/Contender7Audit.lean test/
rm -rf .lake
lake build
lake env lean test/Contender7Audit.lean
```

The package-side checks are:

```
./scripts/run_checks.sh
```

References

[1] V. Reshetnikov et al., *BusyLean: Naming Ever Larger Numbers in Lean*, repository and article, commit `86b360648163f0249eab79539438b6a83dfe8215`, 2026. https://github.com/VladimirReshetnikov/BusyLean

[2] C. Roux, *name-the-biggest-number*, Coq large-number contest repository. https://github.com/codyroux/name-the-biggest-number

[3] The Lean developers, *Lean 4 theorem prover*, version 4.34.0, 2026. https://github.com/leanprover/lean4

[4] H. Schwichtenberg and S. S. Wainer, *Proofs and Computations*, Cambridge University Press, 2012.

[5] J.-Y. Girard, Y. Lafont, and P. Taylor, *Proofs and Types*, Cambridge University Press, 1989.

[6] W. Buchholz, A new system of proof-theoretic ordinal functions, *Annals of Pure and Applied Logic* 32 (1986), 195–207.